

Data Structure

02. 주소와 참조

2주차: 배열과 연속된 메모리

목차 INDEX

01 주소와 참조

- 변수의 구조
- 주소와 참조
- 포인터

02 리스트 (파이썬)

- 파이썬 리스트의 메모리 구조
- 리스트의 동적 확장
- 리스트 슬라이싱

03 복사

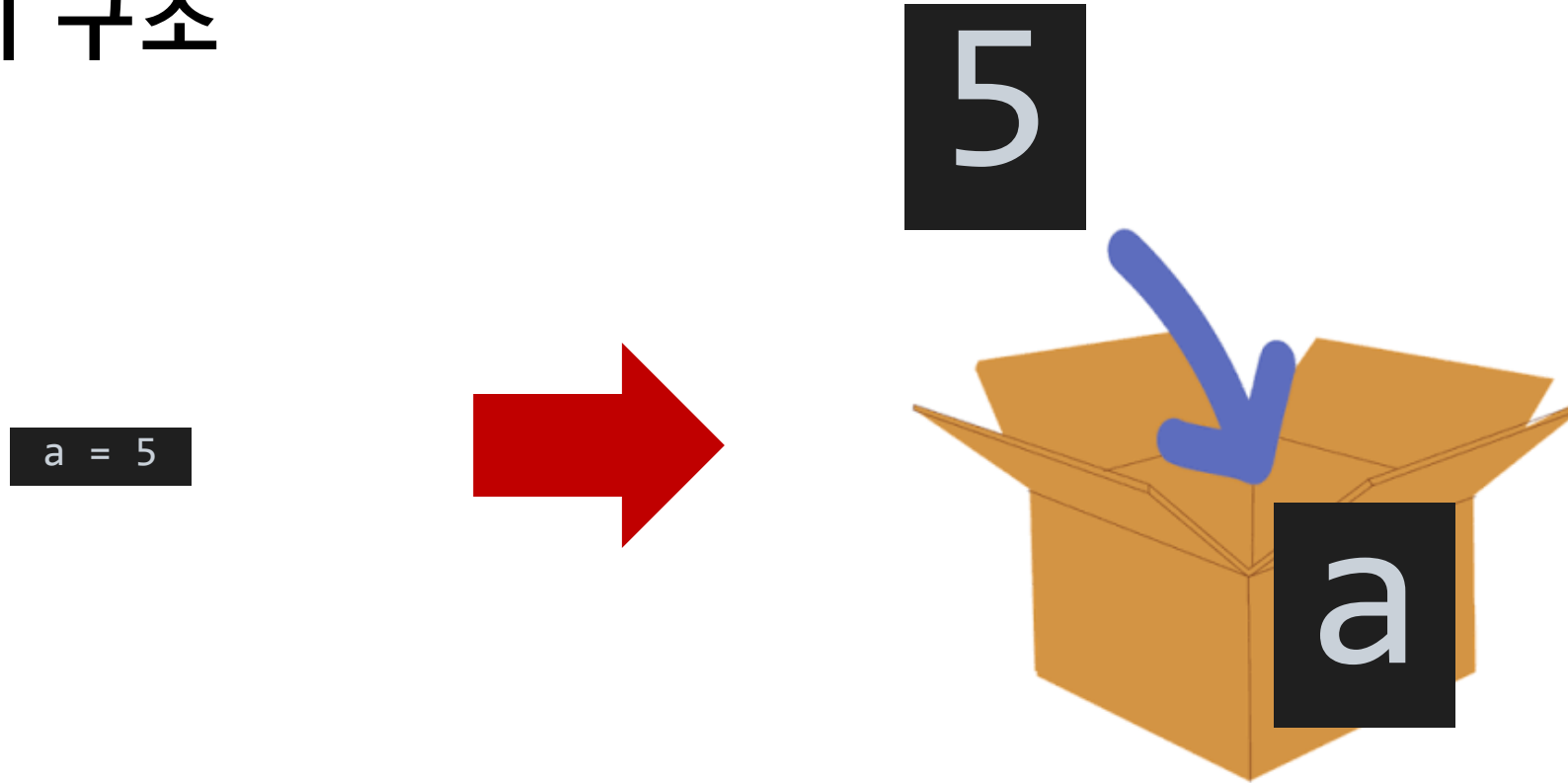
- 얕은 복사
- 깊은 복사

01

주소와 참조

Address & Reference

변수의 실제 구조



변수는 보통 뭘 담을 수 있는 상자로 비유됨.
메모리에 저장된 'a'라는 변수를 까보면 안에 실제로 5가 들어있음
(C, C++가 이런 구조)
그런데 파이썬은 이와 다른 구조를 사용함....

파이썬은 변수의 구조가 다름



파이썬에서는 미리 'int' 클래스로 선언된 '5' 라는 인스턴스를
가리키는 이름으로 'a' 를 사용함
(파이썬에서는 모든 것이 인스턴스임)

C 처럼 박스에 직접 넣으면 안되나요

C, C++의 방식 (정적 타입 언어)

C언어에서는 변수를 선언할 때 무조건 앞에 데이터타입을 붙여야 함

```
int a = 5;
```



```
int a = 5;  
a = "hello";
```



incompatible types

게다가 C언어에서는 각 자료형의 크기가 정해져있음.
그래서 데이터타입마다 메모리에 저장하는 방식이 다 다름

구분	자료형	메모리크기 (byte)
문자형	char	1
	short	2
정수형	int	4

실수형	long	4
	float	4
	double	8

파이썬의 방식 (동적 타입 언어)

파이썬에서는 한 변수에 여러 데이터타입을 선언할 수 있음

```
a = 10    a = 3.14    a = "abc"    a = [1, 2, 3]
```

이게 가능한 이유가 'a'는 그냥 이름표일 뿐이기 때문임.

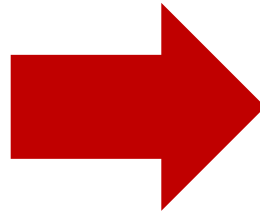
이미 선언된 인스턴스를 가리키는 이름표가 'a'인 것



이거 장점이 뭐임

```
# 천만개의 1이 있는 리스트 만들기  
a = [1] * 10000000  
  
# 그 리스트 복사  
b = a
```

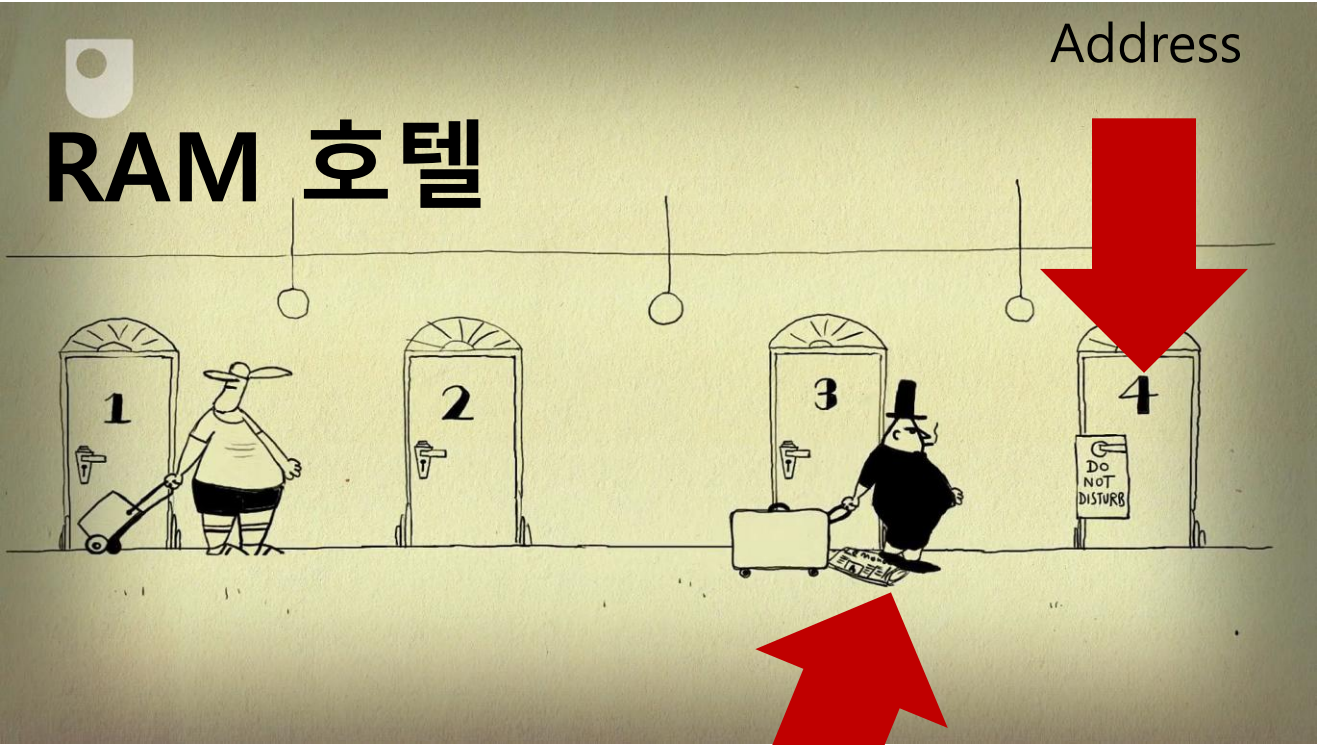
이렇게 큰 리스트 복사할 때 이름표만 복사하면 됨



상자 내용물 모두 복사할 필요 없이
이름표만 하나 더 달면 됨



주소와 참조



"Human" 클래스로 선언된 인스턴스

```
guest_list = ["a", "b", "c", ...]
```

guest_list		
손님 번호	손님 이름	목은 방
0	a	1번 방
1	b	2번 방
2	c	3번 방
⋮	⋮	⋮

↑
인덱스
Index

↑
변수
Variable

↑
참조 / 포인터
Reference / Pointer

02

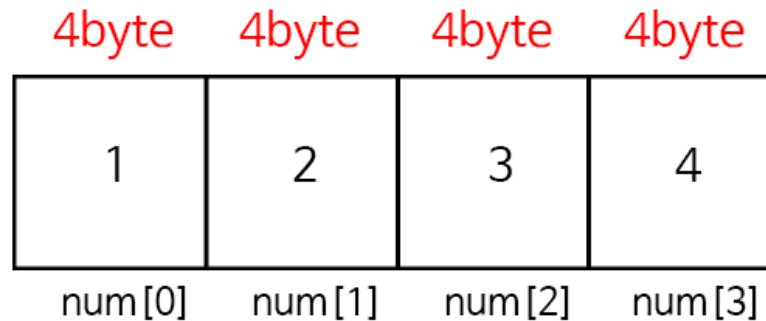
리스트

Python Lists

[복습] C Array의 메모리 구조

- C에서는 배열의 각 원소를 한 칸씩 메모리에 저장함
- 아래는 한 칸이 4바이트인 32비트 컴퓨터에서 각 칸에 int 값들을 저장한 모습임
- C는 배열에 하나의 자료형만 들어갈 수 있기 때문에 이렇게 해도 문제없음
- 아래와 같이 선언함

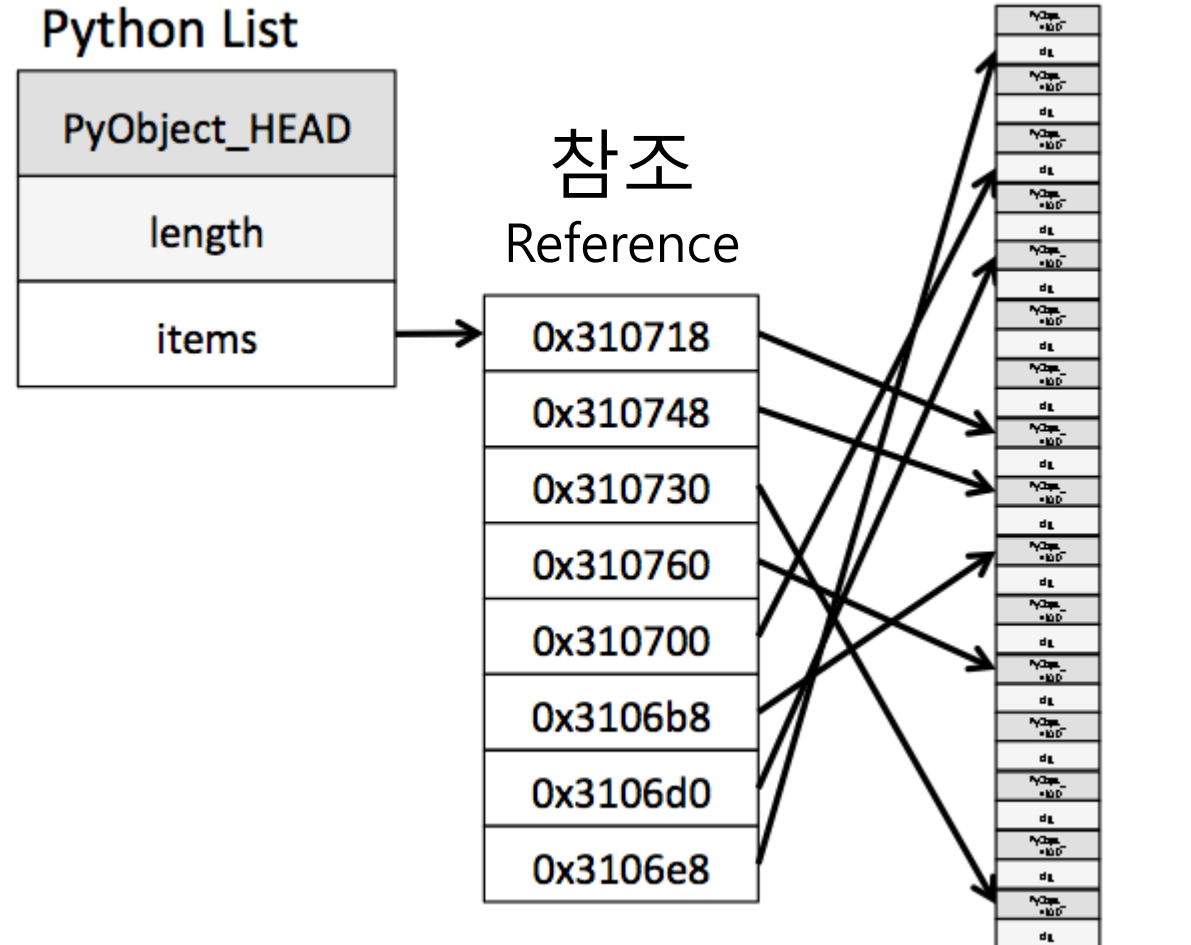
```
int num[4] = {1, 2, 3, 4};
```



Address	Value
0x00	01001010
0x01	10111010
0x02	01011111
0x03	00100100
0x04	01000100
0x05	10100000
0x06	01110100
0x07	01101111
0x08	10111011
...	...
0xFE	11011110
0xFF	10111011

Python List의 메모리 구조

- 파이썬에서는 모든 것이 객체(인스턴스)
(파이썬의 설계 철학임)
- 파이썬은 C와 달리 int, float, str 등 여러 자료형이
한 리스트 안에 들어갈 수 있음
- 그래서 각 원소의 크기가 제각각, C처럼 메모리에
연속해서 원소들을 저장하기 매우 어려움
- 따라서 파이썬은 리스트에 객체를 넣지 않고 참조만
넣음 (객체들을 가리키는 표지판들의 묶음 == list)
- 실제 객체가 메모리의 어디에 저장되어있는지는 몰라
도 됨



타 언어의 Array의 저장 방식

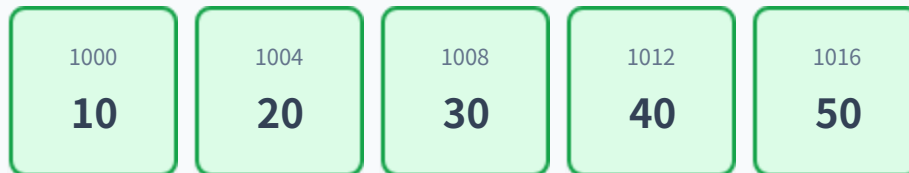
배열의 3대 핵심 특징

- 연속된 메모리 (Contiguous Memory)
데이터가 메모리 상에서 끊김 없이 나란히 배치
- 같은 타입 (Homogeneous)
배열은 같은 크기의 데이터 타입만 저장
- 인덱스 접근 (Index Access)
번호표를 통해 어디든 한 번에 접근 (이건 파이썬도 동일)

메모리 위의 배열 시각화

```
int arr[5] = {10, 20, 30, 40, 50};
```

메모리 주소가 연속적으로 증가함

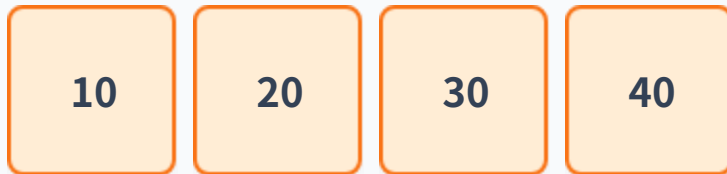


리스트의 동적 확장

⚠ 리스트가 꽉 찼는데 **append()** 할 때

만약 리스트의 여유 공간이 꽉 찬 상태에서 새로운 데이터를 **append()** 하려고 하면?

기존 공간 (꽉 참)



더 큰 공간 새로 할당



리스트의 동적 확장

리스트의 동적 확장

1단계: 빈 리스트를 생성하면 메모리에 참조들을 넣을 여러 칸을 한번에 확보함

2단계: `append()`를 여러 번 진행해 용량이 꽉 차면 더 용량이 큰 빈 리스트를 몰래 생성함

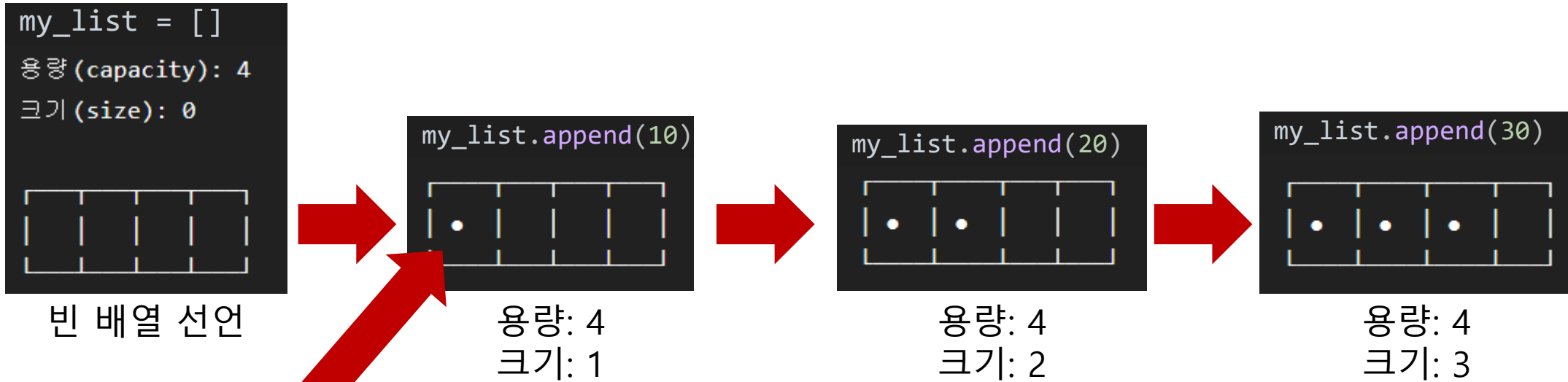
3단계: 기존 리스트에 있던 참조를 모조리 복사함

4단계: 기존에 쓰던 작은 리스트는 갖다 버림

왜 이렇게 함?

- 매번 1칸씩만 늘리면 `append` 할 때마다 빈 리스트를 계속 생성하고 참조를 계속 복사해야함
- 그렇다고 10배씩 늘려버리면 메모리를 낭비하게 됨
- 그래서 파이썬은 현재 길이의 약 12.5% 정도만 추가적으로 확보함

리스트의 동적 확장

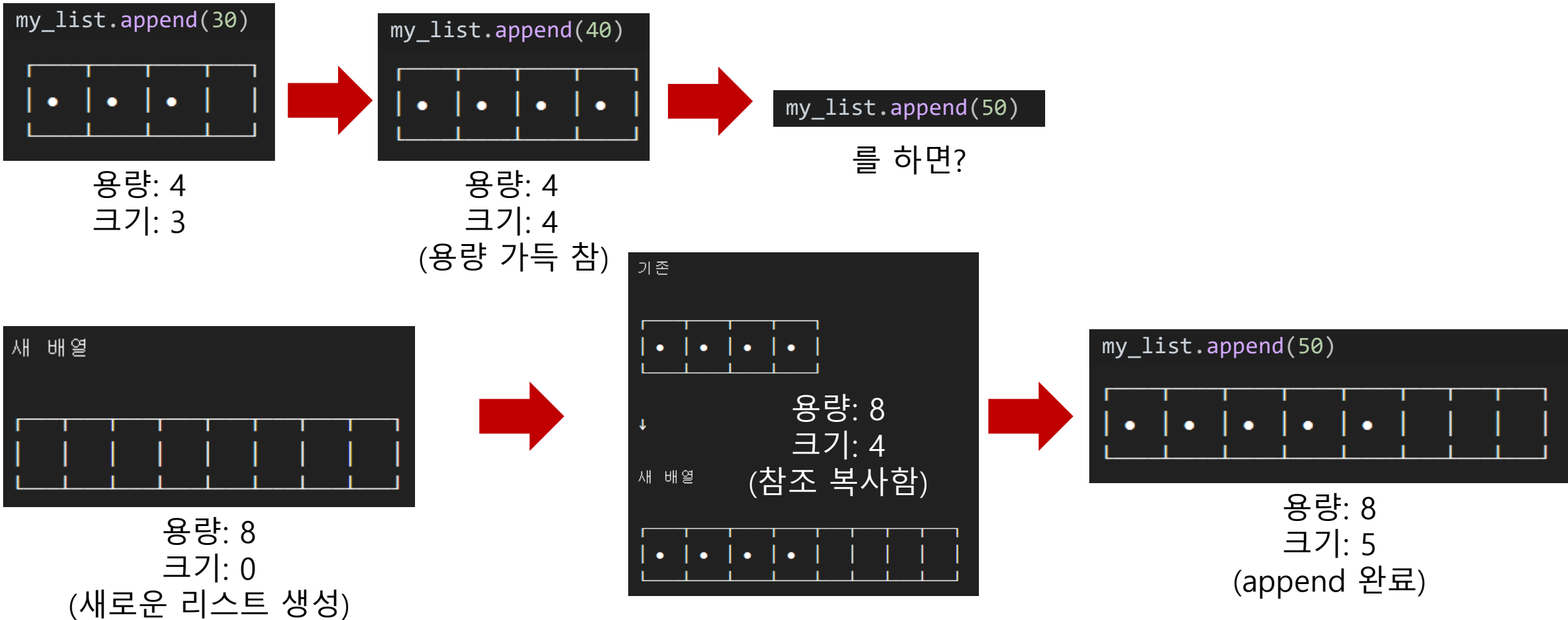


메모리 어딘가에 선언된
10을 가리키는 참조

크기는 `len(my_list)` 를 의미함

용량은 우리가 볼 수 없는
파이썬 내부적인 값임

리스트의 동적 확장



이거 똑같은거 아님?

```
list.append(10)
```

```
list.insert(len(list), 10)
```

스포: 아님



이거 똑같은거 아님?

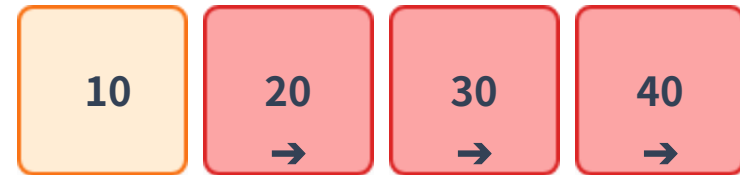
.append()



append() 실행 시 맨 끝에 바로 추가
시간복잡도: $O(1)$

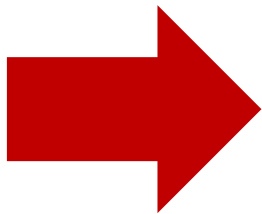
첫 번째 빈공간 위치를 바로 계산해 꽂는 원리

.insert()



뒤에 있는 모든 데이터를 하나씩 뒤로 미는 작업 필요
시간 복잡도: $O(N)$

모든 참조들을 한 칸씩 밀며 꽂는 원리



성능충이 되려면 함수도 가려 써야함

리스트 슬라이싱

? `new_list = arr[:]`

슬라이싱은 기존 리스트를 공유하는 것이 아님

완전히 새로운 리스트 객체를 생성하고 내부 참조들을 싹 다 복사해 가져오는 로직

메모리 관점의 슬라이싱

원본 리스트와는 별개의 새로운 리스트 주소가 할당되며,
내부의 객체 참조들만 그대로 복제

주의할 점

슬라이싱은 새로운 리스트를 만드는 연산이므로 데이터가
아주 많을 때 무분별하게 사용하면 메모리와 시간이
낭비될 수 있음

03

얕은 복사와 깊은 복사

Shallow Copy & Deep Copy

03 얇은 복사와 깊은 복사 Shallow Copy & Deep Copy

이것도 똑같은 거 아님?

```
a = [10, 20, 30]  
b = a[:]
```

```
import copy  
a = [10, 20, 30]  
b = copy.deepcopy(a)
```

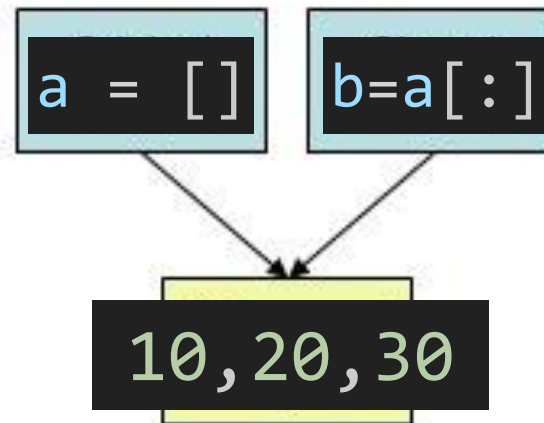
(같지 않다는 대답이 나와야함)



Shallow Copy & Deep Copy

- a 리스트를 선언할 때 메모리 어딘가에 int 클래스의 10, 20, 30이라는 인스턴스가 생성됨
- 그 다음 a 리스트에 그 인스턴스들을 가리키는 포인터들을 담아놓음
- `b = a[:]`로 얇은 복사를 수행하면 b 안에 같은 인스턴스들을 가리키는 포인터만 복사됨
- 반면, 깊은 복사를 수행하면 인스턴스를 새로 생성함
- 따라서 얇은 복사에서는 복사본을 수정하면 원본이 바뀜 (같은 인스턴스를 가리키므로)
그러나 깊은 복사에서는 서로 다른 인스턴스를 가리키므로 복사본을 수정해도 원본은 변하지 않음

Shallow Clone



Deep Clone

